

HEXLASH

Документация боевой системы

Дата: 2026-03-20

Версия: 1.0

Язык: Русский

Оглавление

1 Ядро боевой системы

- 1.1 Общий флоу боя
- 1.2 Механика раунда
- 1.3 AI-стратегия
- 1.4 Генерация противника
- 1.5 Колода и модули

2 Спецмеханики: Dice, Coach, Emergency

- 2.1 Dice (Кубик судьбы)
- 2.2 Coach (Тренер)
- 2.3 Emergency Protocol
- 2.4 Стейт боя (cardFightState)

3 PvP боевой движок

- 3.1 PvP vs PvE — отличия
- 3.2 WebSocket протокол боя
- 3.3 Управление матчем
- 3.4 ELO и рейтинг

4 Матчмейкинг и челленджи

- 4.1 Матчмейкинг
- 4.2 Челленджи (друзья)

5 Auto Fight — автоматические бои

- 5.1 Механика Auto Fight
- 5.2 Офлайн и catch-up
- 5.3 Персистентность
- 5.4 Дневной ресет
- 5.5 AutoFightStatus UI

6 Приложения

- 6.1 Таблица приёмов (18 шт.)
- 6.2 Таблица эффектов Dice
- 6.3 Таблица констант боя
- 6.4 Сводка предупреждений

Раздел 1 — Ядро боевой системы

Ядро боевой системы Hexlash

Документация по исходному коду боевой механики.

Файлы:

- src/core/engine/combatEngine.js — движок боя
- src/core/engine/aiStrategy.js — AI-стратегия (выбор действий)
- src/core/engine/opponentGenerator.js — генерация AI-противника
- src/core/constants.js — константы
- src/data/moves.js — данные приёмов (18 шт.)
- src/data/branches.js — ветки развития (3 шт.)
- src/data/cardPower.js — базовая сила карт для матчмейкинга

1. Общий флоу боя

Константы

Константа	Значение	Описание
`MAX_HP`	100	Стартовое HP каждого бойца
`MAX_ROUNDS`	10	Основные раунды
`EXTRA_ROUNDS`	2	Дополнительные раунды (Overdrive)
`TOTAL_ROUNDS`	12	`MAX_ROUNDS` + `EXTRA_ROUNDS`
`EXTRA_ROUND_DAMAGE_MULTIPLIER`	2	Множитель урона в Overdrive-раундах
`COUNTDOWN`	3	Секунды до начала боя
`ROUND_ANIMATION_MS`	1500	Длительность анимации раунда (мс)

Цикл боя (CombatEngine.runCombat)

1. Оба бойца получают HP = MAX_HP (100)
2. Создаются AI-стратегии: ai1 = new ModuleAIStrategy(modules1), ai2 = new ModuleAIStrategy(modules2)
3. Цикл раундов (1..TOTAL_ROUNDS = 12):
 - a. AI выбирает действие: ai.selectAction(currentHP, MAX_HP)
 - b. Определяется moveInfo — приём из колоды для текущего раунда
 - c. Раунд разрешается через resolveRoundLive()
 - d. HP обновляется

- е. Если HP любого бойца <= 0 → бой прерывается
- 4. buildResult() определяет победителя

Определение победителя (CombatEngine.buildResult)

```
if (finalHP1 <= 0 && finalHP2 <= 0) → НИЧЬЯ
else if (finalHP1 <= 0) → Победа fighter2
else if (finalHP2 <= 0) → Победа fighter1
else if (finalHP1 > finalHP2) → Победа fighter1 (no HP)
else if (finalHP2 > finalHP1) → Победа fighter2 (no HP)
else → НИЧЬЯ (HP равны)
```

Ничья возникает в двух случаях:

- 1. Оба нокаутированы в одном раунде (HP обоих <= 0)
- 2. После 12 раундов HP равны

Если после 12 раундов HP не равны — побеждает боец с большим HP.

Overdrive (раунды 11-12)

Раунды с номером > MAX_ROUNDS (10) считаются Overdrive:

- Весь урон умножается на EXTRA_ROUND_DAMAGE_MULTIPLIER (×2)
- AI получает агрессивный бонус: attack += 30, position = max(5, position - 15)

2. Механика одного раунда

Действия бойца

Каждый раунд боец выбирает одно из трёх действий:

Действие	Эффект
`attack`	Наносит урон противнику
`defense`	Блокирует 60% входящего урона
`position`	Не наносит урон, но даёт `POSITION_BONUS` (+5) к следующей атаке

Пошаговая логика раунда (resolveRoundLive)

- 1. Определяется Overdrive-множитель:
overdriveMult = (roundNum > MAX_ROUNDS) ? 2 : 1
- 2. Рассчитывается урон приёмов:
moveDmg = (moveData.damage ?? BASE_DAMAGE) * overdriveMult
moveSpeed = moveData.speed ?? 1.0

3. Если оба атакуют — определяется порядок по speed:
fighter1First = (moveSpeed1 >= moveSpeed2)
4. Fighter 1 действует:
 - attack → наносит урон
 - position → получает бафф +POSITION_BONUS к следующей атаке
 - defense → ничего активного
5. Fighter 2 действует:
 - Если blindActive → промах (missed)
 - Если shieldActive → урон заблокирован (shield)
 - Иначе → аналогично Fighter 1
6. Speed-based KO: если оба атакуют — более быстрый может нокаутировать до ответного удара медленного

Формула расчёта урона (calcAttackDamage)

- Входные параметры:
- baseDmg — урон приёма из moves.js (с учётом уровня и Overdrive)
 - attackerAi.consumeAttackBoost() — накопленный бонус от position (расходуется)
 - mult — внешний множитель (например, от дайса Adrenaline/Rage)

Атака vs Атака (или vs Position)

```
dmg = (baseDmg + attackerAi.consumeAttackBoost()) * mult
// 10% шанс крита:
if (Math.random() < CRIT_CHANCE) { // 0.10
  finalDmg = Math.floor(dmg * CRIT_MULT) // × 1.5
}
```

⚠ Если защитник в position — есть дополнительный шанс уклонения:

```
dodgeChanceBonus = 0.1 // передаётся в calcAttackDamage
// Итого: 22% шанс уклонения при position
if (Math.random() < 0.12 + 0.1) → dodge (0 урона)
```

Атака vs Защита (defense)

```
dmg = (baseDmg + attackerAi.consumeAttackBoost()) * mult
blocked = Math.floor(dmg * 0.6) // блокируется 60%
finalDmg = Math.max(0, dmg - blocked) // проходит ~40%
```

Пример расчёта

Приём: hook уровня 3, damage = 24, speed = 0.85
Противник в defense, position бонус = 5 (от предыдущего раунда):

```
dmg = (24 + 5) * 1 = 29
blocked = floor(29 * 0.6) = 17
finalDmg = max(0, 29 - 17) = 12
```

Дополнительные механики раунда

Механика	Константа	Значение
----------	-----------	----------

Шанс уклонения (dodge)	`DODGE_CHANCE`	12% (+ 10% бонус при position)
Шанс крита	`CRIT_CHANCE`	10%
Множитель крита	`CRIT_MULT`	×1.5
Блок при defense	—	60% урона блокируется
Position бонус	`POSITION_BONUS`	+5 к следующей атаке

Speed-based KO (нокаут по скорости)

Когда оба бойца атакуют одновременно:

1. Сравниваются speed приёмов: fighter1First = (speed1 >= speed2)

2. Быстрый бьёт первым

3. Если быстрый нокаутирует (HP противника → 0):

→ медленный НЕ наносит ответный удар

→ события урона медленного удаляются из лога

Роль speed приёма

Speed определяет приоритет атаки при обоюдной атаке (attack vs attack):

- Боец с большим speed бьёт первым
- При равном speed — первым бьёт Fighter 1 (игрок)
- Быстрый может нокаутировать до ответного удара
- Speed НЕ влияет на урон, только на порядок ударов

3. AI-стратегия (ModuleAIStrategy)

Архетипы (модули)

Каждый боец имеет 3 модуля-архетипа с весами по слотам:

Слот	Вес
slot1	0.50 (50%)
slot2	0.30 (30%)
slot3	0.20 (20%)

6 архетипов:

ID	Имя	High HP (>70%)	Low HP (<=70%)	Стиль
`predator`	Хищник	atk:80 def:10 pos:10	atk:90 def:5 pos:5	Ультра-агрессивный
`sentinel`	Страж	atk:20 def:50 pos:30	atk:10 def:70 pos:20	Оборонительный

`ghost`	Призрак	atk:30 def:20 pos:50	atk:40 def:10 pos:50	Позиционный
`analyst`	Аналитик	atk:35 def:30 pos:35	atk:30 def:30 pos:40	Сбалансированный
`maverick`	Непредсказуемый	atk:33 def:33 pos:34	**random**	Хаотичный
`juggernaut`	Неостановимый	atk:50 def:35 pos:15	atk:50 def:35 pos:15	Агрессивно-стабильный

Расчёт приоритетов (calculatePriorities)

```
hpPercent = (currentHP / maxHP) * 100
hpState = hpPercent > 70 ? 'high' : 'low'

// Взвешенная сумма приоритетов трёх модулей:
combined.attack   = module1.attack * 0.5 + module2.attack * 0.3 + module3.attack * 0.2
combined.defense   = module1.defense * 0.5 + module2.defense * 0.3 + module3.defense * 0.2
combined.position  = module1.position * 0.5 + module2.position * 0.3 + module3.position * 0.2

// Если есть коуч-буст:
combined[coachBoost.action] += 25
```

Maverick при low HP

Особый случай — при hpState = 'low' приоритеты = 'random':

```
spike = Math.random() < 0.3 // 30% шанс "вспышки"
if (spike) {
  // Один случайный тип действия получает 80
  priorities = { attack: 33, defense: 33, position: 34 }
  priorities[randomType] = 80
} else {
  priorities = { attack: 33, defense: 33, position: 34 } // равномерно
}
```

Выбор действия (selectAction)

```
// В Overdrive: attack += 30, position = max(5, position - 15)
total = attack + defense + position
roll = Math.random() * total

if (roll < attack)      → 'attack'
if (roll < attack + defense) → 'defense'
else                    → 'position'
```

Пример: predator + sentinel + ghost (high HP)

```
attack: 80*0.5 + 20*0.3 + 30*0.2 = 40 + 6 + 6 = 52
defense: 10*0.5 + 50*0.3 + 20*0.2 = 5 + 15 + 4 = 24
position: 10*0.5 + 30*0.3 + 50*0.2 = 5 + 9 + 10 = 24
total = 100
```

→ 52% attack, 24% defense, 24% position

Dice preferences (дайс-предпочтения)

AI решает, подбирать ли дайс-предмет через shouldPickupDiceItem:

```
combinedChance = module1.pref * 0.5 + module2.pref * 0.3 + module3.pref * 0.2
pickup = Math.random() * 100 < combinedChance
```

Эффекты дайсов:

- heal — лечение
- shield — блокирует следующую атаку противника
- blind — противник промахивается следующей атакой
- rage — множитель атаки
- crit — критический удар
- adrenaline — множитель атаки

Коуч (Coach Boost)

```
// Активируется с раунда COACH_MIN_ROUND (6)
// Шанс срабатывания: COACH_TRIGGER_CHANCE (1.0 = 100%)
// Длительность: COACH_BOOST_ROUNDS (4 раунда)
// Эффект: +25 к приоритету выбранного действия
```

4. Генерация противника (OpponentGenerator)

Параметры генерации

```
OpponentGenerator.generate(difficulty, playerPower)
// difficulty: 'easy' | 'medium' | 'hard'
// playerPower: силовой рейтинг игрока (опционально)
```

Что рандомизируется

Параметр	Источник
Имя	Случайное из 20 имён (Shadow, Viper, Thunder...) + префикс по сложности
Скин	Случайный из 3 скинов (`skin_m_1/2/3.png`)

Модули (3 шт.)	Случайные архетипы без повторов из пула сложности
Колода	Генерируется через `generateDeckForPower()` под силу игрока
Уровни карт	Рассчитываются для соответствия целевой силе

Пулы архетипов по сложности

Сложность	Префикс	Предпочтительные архетипы
`easy`	"Rookie "	analyst, sentinel, maverick
`medium`	" "	все 6 архетипов
`hard`	"Elite "	predator, juggernaut, ghost, analyst

Power-based матчмейкинг

Если передан playerPower:

```
// Целевая сила противника зависит от сложности:
// easy:  playerPower * [0.70 .. 0.85]
// medium: playerPower * [0.90 .. 1.10]
// hard:  playerPower * [1.15 .. 1.30]

targetPower = clamp(randomInRange, MIN_OPPONENT_POWER=25, MAX_OPPONENT_POWER=200)
```

Генерация колоды (generateDeckForPower)

```
deckSize = random(3, 5)

// Бюджет силы на карты:
cardsPowerBudget = targetPower - MODULE_BASE_POWER(15) - bonusPower(5)
powerPerCard = cardsPowerBudget / deckSize

// Для каждой карты:
// Выбирается случайный приём (без повторов)
// Рассчитывается уровень:
level = round(((powerPerCard / basePower) - 1) / 0.2 + 1)
level = clamp(level, 1, 5)
```

5. Колода и модули

Ограничения колоды

Константа	Значение
`MIN_DECK_SIZE`	4
`MAX_DECK_SIZE`	8

Выбор приёма из колоды в бою

В каждом раунде приём определяется циклически:

```
moveId = deck[(roundNum - 1) % deck.length]
```

Колода из 5 карт в бою на 12 раундов:

```
Индекс: 0 1 2 3 4 0 1 2 3 4 0 1
```

Раунд

Уровень приёма (1-5)

Уровень берётся из cardLevels[moveId] и влияет на значения damage и speed из массивов moves.js:

```
const level = clamp(cardLevels[moveId] || 1, 1, 5)
damage = moveData.damage[level - 1]
speed = moveData.speed[level - 1]
```

cardPower.js — система силового рейтинга

Используется для матчмейкинга, НЕ для боевых расчётов.

Формула силового рейтинга (calculatePowerRating):

```
power = 0

// 1. Сила карт в колоде
deck.forEach(cardId => {
  power += CARD_BASE_POWER[cardId] * LEVEL_MULTIPLIERS[level]
})

// 2. Сила модулей (с позиционными весами)
modules.forEach((moduleId, index) => {
  power += MODULE_BASE_POWER(15) * POSITION_WEIGHTS[index]
  // POSITION_WEIGHTS = [0.5, 0.3, 0.2]
  // Итого от модулей: 15 * (0.5 + 0.3 + 0.2) = 15
})

// 3. Бонус за открытые карты (+1 за каждую)
power += unlockedCards.length

return Math.round(power)
```

Базовая сила карт (CARD_BASE_POWER):

Ветка	Карты (от слабой к сильной)
-------	-----------------------------

Speed	jab:8, double_jab:10, rapid_fire:12, combo_strike:14, flurry:16, hurricane:18
Power	straight:10, hook:13, uppercut:16, haymaker:19, hammer_fist:22, knockout_blow:25
Technique	block_strike:9, counter_jab:11, feint_cross:13, parry_punish:15, slip_counter:17, precision_strike:20

Множители уровня (LEVEL_MULTIPLIERS):

Уровень	Множитель
1	×1.0
2	×1.2
3	×1.4
4	×1.6
5	×1.8

6. Таблица приёмов

Speed (Скорость)

ID	Damage (1→5)	Speed (1→5)
`jab`	8 → 10 → 12 → 15 → 18	1.20 → 1.30 → 1.40 → 1.50 → 1.60
`double_jab`	12 → 15 → 18 → 22 → 26	1.10 → 1.20 → 1.30 → 1.40 → 1.50
`rapid_fire`	15 → 18 → 22 → 27 → 32	1.00 → 1.10 → 1.20 → 1.30 → 1.40
`combo_strike`	20 → 24 → 29 → 35 → 42	0.90 → 1.00 → 1.10 → 1.20 → 1.30
`flurry`	25 → 30 → 36 → 43 → 52	0.80 → 0.90 → 1.00 → 1.10 → 1.20
`hurricane`	32 → 38 → 46 → 55 → 66	0.70 → 0.80 → 0.90 → 1.00 → 1.10

Power (Сила)

ID	Damage (1→5)	Speed (1→5)
----	--------------	-------------

`straight`	12 → 15 → 18 → 22 → 27	0.80 → 0.85 → 0.90 → 0.95 → 1.00
`hook`	16 → 20 → 24 → 29 → 35	0.75 → 0.80 → 0.85 → 0.90 → 0.95
`uppercut`	20 → 25 → 30 → 36 → 44	0.70 → 0.75 → 0.80 → 0.85 → 0.90
`haymaker`	26 → 32 → 38 → 46 → 56	0.60 → 0.65 → 0.70 → 0.75 → 0.80
`hammer_fist`	32 → 40 → 48 → 58 → 70	0.50 → 0.55 → 0.60 → 0.65 → 0.70
`knockout_blow`	42 → 52 → 62 → 75 → 90	0.40 → 0.45 → 0.50 → 0.55 → 0.60

Technique (Техника)

ID	Damage (1→5)	Speed (1→5)
`block_strike`	10 → 12 → 15 → 18 → 22	1.00 → 1.05 → 1.10 → 1.15 → 1.20
`counter_jab`	14 → 17 → 21 → 25 → 30	0.95 → 1.00 → 1.05 → 1.10 → 1.15
`feint_cross`	18 → 22 → 27 → 32 → 39	0.90 → 0.95 → 1.00 → 1.05 → 1.10
`parry_punish`	22 → 27 → 33 → 40 → 48	0.85 → 0.90 → 0.95 → 1.00 → 1.05
`slip_counter`	28 → 34 → 41 → 50 → 60	0.80 → 0.85 → 0.90 → 0.95 → 1.00
`precision_strike`	35 → 43 → 52 → 63 → 76	0.75 → 0.80 → 0.85 → 0.90 → 0.95

Закономерности

- Внутри ветки: чем сильнее приём, тем выше урон, но ниже скорость
- Между ветками (на одном уровне сложности): Power > Technique > Speed по урону, Speed > Technique > Power по скорости
- Прокачка (уровни 1→5): урон растёт примерно ×2-2.2, скорость растёт примерно на +0.20

Раздел 2 — Dice, Coach, Emergency

Dice, Coach, Emergency — спецмеханики боя

Документация по исходному коду: дайс (кубик), тренер (коуч), экстренный протокол, стейт боя.

Файлы:

- src/core/state/modules/cardFightState.js — Vuex-модуль боя (логика dice, coach, emergency, persistence)
- src/views/CardFightView.vue — UI боя (отображение dice/coach/emergency)
- src/core/engine/combatEngine.js — движок (применение модификаторов)
- src/core/constants.js — константы

1. Dice (Кубик судьбы)

Общее

Игрок может вручную бросить кубик во время боя. Кубик выдаёт случайный эффект из 6 возможных.

Константы:

Константа	Значение	Описание
`DICE_COOLDOWN_ROUNDS`	3	Раунды кулдауна после броска

Когда Dice доступен

UI-условие отображения кнопки (CardFightView.vue:85):

```
&& roundNum > 0
&& !isOverdrive
&& (diceState.ready || diceState.activeItem)
```

- Доступен с 1 раунда
- Недоступен в Overdrive (раунды 11-12)
- После использования — кулдаун 3 раунда
- Кулдаун тикает в computeNextRound после каждого раунда

Все 6 эффектов

Реализация в cardFightState.js, action rollDiceManual:

ID	Emoji	Эффект	Реализация в коде	Длительность
`heal`		Лечение	`HP1 = min(MAX_HP, HP1 + 15)`	Мгновенно

`adrenaline`	⚡	Двойная атака	`attackMultiplier = 2`	1 раунд (сбрасывается после `resetPlayerModifiers`)
--------------	---	---------------	------------------------	--

`shield`		Блокирует атаку противника	`shieldActive = true`	1 раунд
----------	--	----------------------------	-----------------------	---------

`blind`		Противник промахивается	`blindActive = true`	1 раунд
---------	--	-------------------------	----------------------	---------

`rage`		Мгновенный урон	`HP2 -= 20` (может нокаутировать)	Мгновенно
--------	--	-----------------	-----------------------------------	-----------

`crit`		Мгновенный крит-урон	`HP2 -= 30` (может нокаутировать)	Мгновенно
--------	--	----------------------	-----------------------------------	-----------

Как dice выбирается

```
// cardFightState.js:446 – равновероятный случайный выбор
const item = DICE_ITEMS[Math.floor(Math.random() * DICE_ITEMS.length)];
// Вероятность каждого = 1/6 ≈ 16.7%
```

Как dice влияет на расчёт урона

Модификаторы playerModifiers передаются в CombatEngine.resolveRoundLive как playerMods:

```
// combatEngine.js:40-45
const {
  attackMultiplier = 1, // от adrenaline dice или emergency
  shieldActive      = false, // от shield dice или emergency
  blindActive       = false, // от blind dice
} = playerMods;
```

Применение в движке:

- attackMultiplier — множитель mult при вызове calcAttackDamage для Fighter 1 (combatEngine.js:92)
- shieldActive — если противник атакует, урон = 0, событие shield (combatEngine.js:106-107)
- blindActive — если противник атакует, урон = 0, событие missed (combatEngine.js:104-105)

⚠ Модификаторы применяются только к Fighter 1 (игроку). Fighter 2 (AI) всегда атакует с mult = 1 и без shield/blind.

Сброс модификаторов

```
// cardFightState.js:126 – после каждого раунда
commit('resetPlayerModifiers');
// → { attackMultiplier: 1, shieldActive: false, blindActive: false }
```

Все dice-эффекты (кроме heal, rage, crit) действуют ровно 1 раунд.

Кулдаун

```
// cardFightState.js:371-373 – тик кулдауна после каждого раунда
if (state.diceState.cooldownLeft > 0) {
  const newCd = state.diceState.cooldownLeft - 1;
  commit('setDiceState', { cooldownLeft: newCd, ready: newCd <= 0 });
}
```

Цикл: бросок → кулдаун 3 раунда → снова доступен → бросок → ...

AI и Dice

AI-противник не бросает кубик в PvE бою. Метод shouldPickupDiceItem из aiStrategy.js существует, но в cardFightState.js не вызывается для AI-противника.

△ shouldPickupDiceItem используется только в PvP-контексте (серверная логика). В PvE-бою dice — привилегия игрока.

Auto-fight Dice

При включённом автобое кубик бросается автоматически (CardFightView.vue:684-691):

```
watch([() => diceState.value.ready, fightPhase], ([ready, phase]) => {
  if (isAutoFightEnabled.value && ready && phase === 'fighting' && roundNum.value > 0) {
    setTimeout(() => {
      store.dispatch('fight/rollDiceManual');
    }, 800); // задержка 800ms
  }
});
```

Overdrive: dice отключён

В Overdrive (раунды 11-12):

- UI не показывает кнопку dice (!isOverdrive)
- rollDiceManual проверяет: if (state.roundNum > MAX_ROUNDS) return;
- computeNextRound: модификаторы принудительно сбрасываются:

```
const modsToUse = isOverdrive
  ? { attackMultiplier: 1, shieldActive: false, blindActive: false }
  : state.playerModifiers;
```

2. Coach (Тренер)

Условия активации

```
// cardFightState.js:388
if (!state.coachAdvice.used
    && state.roundNum >= COACH_MIN_ROUND      // >= 6
    && Math.random() < COACH_TRIGGER_CHANCE)  // < 1.0 (= 100%)
```

Константа	Значение	Описание
`COACH_MIN_ROUND`	6	Минимальный раунд для срабатывания
`COACH_TRIGGER_CHANCE`	1.0	Шанс срабатывания (100%)
`COACH_BOOST_ROUNDS`	4	Длительность бонуса в раундах

Итого: Coach всегда срабатывает ровно на раунде 6 (chance = 100%, used = false).
Срабатывает один раз за бой (флаг used).

Что происходит при активации

- 1. Бой ставится на паузу: fightPhase меняется на 'coach'
- 2. Таймер боя останавливается (stopFightTimer())
- 3. Показывается оверлей с 3 кнопками выбора
- 4. Запускается таймер обратного отсчёта: 15 секунд (adviceTimer = 15)
- 5. Если время истекло — coach автоматически пропускается (skipCoachAdvice)

UI оверлея Coach

Полноэкранный оверлей (CardFightView.vue:120-152) с тремя опциями:

Кнопка	Action	Цвет	Описание
Attack	`'attack'`	Красный (#e74c3c)	Бонус к атаке
Defense	`'defense'`	Синий (#3498db)	Бонус к защите
Position	`'position'`	Фиолетовый (#9b59b6)	Бонус к позиционированию

Механика бонуса

При выборе (applyCoachAdvice):

```
// cardFightState.js:486-489
if (_ail) _ail.setCoachBoost(action, COACH_BOOST_ROUNDS); // 4 раунда
commit('setCoachAdvice', { used: true, active: true, action, roundsLeft: COACH_BOOST_ROUNDS });
commit('setFightPhase', 'fighting'); // бой продолжается
```

В AI-стратегии (aiStrategy.js:53-55):

```
// +25 к приоритету выбранного действия
if (this.coachBoost && this.coachBoost.roundsLeft > 0) {
  combined[this.coachBoost.action] += 25;
}
```

Пример: Если AI игрока имеет приоритеты { attack: 52, defense: 24, position: 24 } и выбран coach 'attack':

defense: 24	→ 24/125 = 19.2%
position: 24	→ 24/125 = 19.2%

Тик Coach Boost

Каждый раунд (computeNextRound):

```
if (state.coachAdvice.active && state.coachAdvice.roundsLeft > 0) {
  const newLeft = state.coachAdvice.roundsLeft - 1;
  if (_ail) _ail.tickCoachBoost(); // уменьшает roundsLeft в AI
  if (newLeft <= 0) {
    commit('setCoachAdvice', { active: false, roundsLeft: 0, action: null });
  } else {
    commit('setCoachAdvice', { roundsLeft: newLeft });
  }
}
```

UI индикатор активного Coach

Когда буст активен (CardFightView.vue:112-116):

```

<span>{{ coachActionLabel }}</span> <!-- "ATK +", "DEF +", или "POS +" -->
<span>{{ coachAdvice.roundsLeft }}</span> <!-- оставшиеся раунды -->
</div>
```

При пропуске Coach

```
// cardFightState.js:492-495
skipCoachAdvice({ commit }) {
  commit('setCoachAdvice', { used: true, active: false, action: null, roundsLeft: 0 });
  commit('setFightPhase', 'fighting');
}
```

Coach помечается как used — повторно не появится.

Auto-fight Coach

При автобое coach обрабатывается автоматически (CardFightView.vue:614-618):

```
const autoAction = liveHP1.value < 50 ? 'defense' : 'attack';
setTimeout(() => {
  store.dispatch('fight/applyCoachAdvice', autoAction);
}
```

```
    }, 500);  
  }
```

Стратегия автобоя: HP < 50 → defense, иначе → attack.

Overdrive: coach отключён

В Overdrive (раунды 11-12) блок coach/dice/emergency пропускается:

```
if (!isOverdrive) {  
    // Check Emergency, Dice, Coach  
}
```

3. Emergency Protocol (Экстренный протокол)

Общее

Одноразовая механика спасения. Игрок до боя выбирает тип экстренного протокола. Срабатывает автоматически при определённых условиях.

Состояние

```
// cardFightState.js:158  
emergencyProtocol: { type: 'medkit', used: false }
```

- type — выбранный протокол: 'medkit' | 'adrenaline' | 'shield'
- used — использован ли (одноразовый за бой)
- По умолчанию: 'medkit'

Три протокола

Протокол	Условие срабатывания	Эффект	Код
`medkit`	HP игрока < 30%	Лечение +25 HP	`HP1 = min(100, HP1 + 25)`
`adrenaline`	HP игрока < 20% **И** HP	Двойная атака	`attackMultiplier = 2`
	противника < 30%		
`shield`	Последние 3 раунда — игрок	Блок следующей атаки	`shieldActive = true`
	получал урон каждый раунд		

Реализация (checkEmergencyProtocol)

```
// cardFightState.js:401-437
checkEmergencyProtocol({ commit, state }) {
  if (state.emergencyProtocol.used) return;

  const hpPercent = (state.liveHP1 / MAX_HP) * 100;
  let shouldTrigger = false;

  switch (protocol.type) {
    case 'medkit':
      shouldTrigger = hpPercent < EMERGENCY_HP_THRESHOLD; // < 30
      break;
    case 'adrenaline':
      shouldTrigger = hpPercent < 20
        && (state.liveHP2 / MAX_HP) * 100 < 30;
      break;
    case 'shield': {
      const lastRounds = state.roundLog.slice(-3);
      shouldTrigger = lastRounds.length >= 3
        && lastRounds.every(r => r.damage1 > 0);
      break;
    }
  }

  if (shouldTrigger) {
    // Применить эффект...
    commit('setEmergencyUsed', true);
    commit('setEventTitle', {
      title: t.value.fight.lblEventEmergency,
      cls: 'event-emergency',
      image: PROTOCOL_IMAGES[protocol.type]
    });
  }
}
```

Порядок проверки в раунде

computeNextRound:	
1. _simulateOneRound()	— раунд отыгрывается
2. checkEmergencyProtocol()	— проверка емергенсу (после урона)
3. Tick dice cooldown	— тик кулдауна кубика
4. Tick coach boost	— тик буста тренера
5. Check coach trigger	— проверка появления тренера

Emergency проверяется после получения урона в раунде, что позволяет среагировать на только что полученный урон.

Связь с колодой/билдом

⚠ Emergency protocol не связан с колодой или модулями. Тип протокола задаётся через setEmergencyProtocol action и хранится в стейте. Выбор делается на экране подготовки к бою (PreparationView).

Overdrive: emergency отключён

Как и dice/coach — emergency не проверяется в Overdrive раундах.

Emergency-эффекты vs Dice-эффекты

Свойство	Emergency medkit	Dice heal
Лечение	+25 HP	+15 HP
Триггер	Автоматически при HP < 30%	Ручной бросок

Разовость	1 раз за бой	Каждые 3 раунда
-----------	--------------	-----------------

Свойство	Emergency adrenaline	Dice adrenaline
Эффект	`attackMultiplier = 2`	`attackMultiplier = 2`

Условие	HP < 20% И HP противника < 30%	Случайный бросок
---------	--------------------------------	------------------

Свойство	Emergency shield	Dice shield
Эффект	`shieldActive = true`	`shieldActive = true`

Условие	3 раунда подряд получал урон	Случайный бросок
---------	------------------------------	------------------

4. Стейт боя (cardFightState)

Vuex-модуль fight

Namespace: fight (namespaced: true)

Все данные стейта

```
// cardFightState.js:148-180
{
  playerModules: ['predator', 'analyst', 'ghost'], // 3 архетипа
  opponent: null,                                // объект противника

  playerDeck: [],                                // массив move ID
  playerCardLevels: {},                          // { moveId: level }
  opponentDeck: [],
  opponentCardLevels: {},

  emergencyProtocol: { type: 'medkit', used: false },

  liveHP1: 100,                                  // HP игрока
  liveHP2: 100,                                  // HP противника
  roundNum: 0,                                    // текущий раунд
```

```
roundLog: [], // массив RoundResult

playerModifiers: {
  attackMultiplier: 1,
  shieldActive: false,
  blindActive: false
},
diceState: {
  activeItem: null, // текущий выброшенный предмет
  cooldownLeft: 0, // раунды до следующего броска
  ready: true // можно ли бросать
},
coachAdvice: {
  used: false, // использован ли за бой
  active: false, // активен ли буст
  action: null, // 'attack'|'defense'|'position'
  roundsLeft: 0 // оставшиеся раунды буста
},

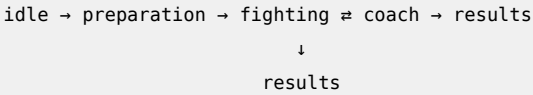
eventTitle: null, // текст события (dice pickup, emergency)
eventTitleClass: '', // CSS-класс события
eventImage: null, // иконка события

fightPhase: 'idle', // idle | preparation | fighting | coach | results
difficulty: 'medium', // easy | medium | hard

fightStats: {
  totalDamageDealt: 0,
  totalDamageTaken: 0,
  dicePickedUp: 0,
  diceIgnored: 0,
  criticalHits: 0
},

xpEarned: null, // { speed, power, technique } или null
xpAwarded: false // предотвращает двойное начисление XP
}
```

Фазы боя (fightPhase)



Фаза	Описание
`idle`	Начальное состояние
`preparation`	Выбор модулей и подготовка
`fighting`	Активный бой (раунды тикают)
`coach`	Пауза: ожидание выбора тренера

<code>`results`</code>	Бой завершён, показ результатов
------------------------	---------------------------------

Что персистится в localStorage

Ключ: `hexlash_current_fight`

Функция `saveFightState` сохраняет (`cardFightState.js:36-61`):

Поле	Описание
<code>`playerModules`</code>	3 архетипа игрока
<code>`opponent`</code>	Полный объект противника
<code>`playerDeck`, `playerCardLevels`</code>	Колода и уровни игрока

<code>`opponentDeck`, `opponentCardLevels`</code>	Колода и уровни противника
---	----------------------------

<code>`liveHP1`, `liveHP2`</code>	Текущие HP
<code>`roundNum`</code>	Текущий раунд
<code>`roundLog`</code>	Лог всех раундов
<code>`playerModifiers`</code>	Активные модификаторы
<code>`diceState`</code>	Состояние кубика (без анимации)

<code>`coachAdvice`</code>	Состояние тренера
<code>`emergencyProtocol`</code>	Тип и статус использования
<code>`fightStats`</code>	Статистика боя
<code>`fightPhase`</code>	Текущая фаза
<code>`difficulty`</code>	Сложность
<code>`xpEarned`, `xpAwarded`</code>	XP данные
<code>`lastUpdateAt`</code>	Timestamp последнего обновления

△ При сохранении `dice`-анимация сбрасывается: `activeItem` всегда `null`, `ready` вычисляется из `cooldownLeft`.

Отдельно в localStorage

Ключ	Данные
<code>`hexlash_player_modules`</code>	Массив из 3 ID архетипов игрока

Восстановление боя при перезагрузке (initFromStorage)

Вызывается при монтировании `CardFightView` (`cardFightState.js:503-568`):

1. Загрузить JSON из `localStorage`
2. Если `fightPhase` = `'idle'` или `'preparation'` → очистить, вернуть `false`

```

3. Восстановить все поля стейта через commit
4. Если fightPhase = 'results':
    → показать результаты, AI не нужен
5. Пересоздать AI-экземпляры:
    → _ai1 = new ModuleAIStrategy(saved.playerModules)
    → _ai2 = new ModuleAIStrategy(saved.opponent.modules)
6. Восстановить coach boost в AI, если был активен
7. Если fightPhase = 'coach':
    → показать оверлей тренера, ждать выбора
8. Если fightPhase = 'fighting':
    → досимулировать пропущенные раунды:
    missedRounds = min(
        floor((Date.now() - lastUpdateAt) / ROUND_ANIMATION_MS),
        TOTAL_ROUNDS - savedRoundNum
    )
    → для каждого пропущенного — _simulateOneRound()

```

Досимуляция при возврате на вкладку

CardFightView.vue слушает visibilitychange:

```

// CardFightView.vue:587-590
if (document.visibilityState === 'visible' && fightPhase.value === 'fighting') {
    store.dispatch('fight/resumeMissedRounds');
}

```

resumeMissedRounds (cardFightState.js:575-590):

```

const missedRounds = min(
    floor(elapsed / ROUND_ANIMATION_MS), // 1500ms per round
    TOTAL_ROUNDS - state.roundNum
);
// Досимулировать каждый пропущенный раунд

```

⚠ Досимулированные раунды не показывают dice/coach/emergency — используется _simulateOneRound, который не вызывает эти механики. Это значит: пропущенные раунды = чистый бой без спецмеханик.

Предупреждение при уходе со страницы

```

// CardFightView.vue:594-598
if (fightPhase === 'fighting' || fightPhase === 'coach') {
    e.preventDefault(); // стандартное предупреждение браузера
    e.returnValue = '';
}

```

Раздел 3 — PvP боевой движок

PvP боевой движок

Документация серверного PvP движка и его отличий от клиентского PvE.

Файлы:

- backend/src/services/pvpCombatEngine.js — серверный движок PvP
- backend/src/services/pvpMatchManager.js — менеджер матчей
- backend/src/websocket/pvpHandler.js — обработка WS-сообщений PvP
- backend/src/config.js — серверные константы
- src/core/state/modules/pvpState.js — клиентский Vuex-модуль PvP

1. PvP движок vs PvE

Архитектурное отличие

Аспект	PvE (клиент)	PvP (сервер)
Где работает	Браузер (`combatEngine.js`)	Node.js (`pvpCombatEngine.js`)
Модуль	ES Module (import/export)	CommonJS (require)
AI	`ModuleAIStrategy` выбирает action	Нет AI — оба игрока реальные
Действия	3 действия: attack/defense/position	⚠ **Нет выбора действий** — оба всегда атакуют
Расчёт урона	`calcAttackDamage` с dodge/crit/block	Прямой урон из `moves.js` + эффекты dice
Dice	Ручной бросок, 6 эффектов на 1 раунд	Ручной бросок, эффекты с **длительностью** (1-2 раунда)
Coach	Игрок выбирает буст действия (+25 приоритет)	Оба игрока решают принять/отклонить, буст = эффект `adrenaline` на 4 раунда

Emergency	Автоматический	протокол	△ **Не реализован** в PvP движке
	(medkit/adrenaline/shield)		
Таймер раунда	ROUND_ANIMATION_MS` (1500ms)	между	ROUND_ANIMATION_MS` (1500ms) через
	раундами`setTimeout`		
Persistence	localStorage	PostgreSQL (Prisma)	

Ключевые расхождения

△ Нет системы действий (attack/defense/position)

В PvE каждый раунд AI выбирает attack, defense или position. В PvP движке нет никакого выбора действий — оба игрока автоматически атакуют каждый раунд. Раунд = обмен ударами приёмами из колод.

```
// pvpCombatEngine.js:127-231 - simulateRound
// Нет вызова selectAction(), нет action1/action2
// Сразу damage из moveData:
let damage1 = moveData1.damage[level1 - 1] * overdriveMult;
let damage2 = moveData2.damage[level2 - 1] * overdriveMult;
```

△ Другая формула урона

PvE: (baseDmg + attackBoost) * mult с шансами dodge (22%), crit (10%), block (60%)

PvP: прямой урон из moves.js + множители от активных эффектов dice, без dodge/crit/block

```
// pvpCombatEngine.js:169-170
damage1 = this.applyEffects(damage1, this.player1, this.player2);
damage2 = this.applyEffects(damage2, this.player2, this.player1);
```

△ Dice эффекты отличаются

Эффект	PvE (клиент)	PvP (сервер)
`heal`	+15 HP, мгновенно	+20 HP, мгновенно
`adrenaline`	`attackMultiplier = 2` (1 раунд)	×1.3 урона, **2 раунда**
`shield`	Полный блок 1 атаки (1 раунд)	×0.5 входящего урона, **2 раунда**
`blind`	Противник промахивается (1 раунд)	50% шанс промаха, **2 раунда**
`rage`	Мгновенно -20 HP противнику	×1.5 урона, **2 раунда**
`crit`	Мгновенно -30 HP противнику	×2 урона, **1 раунд**

△ Coach работает иначе

PvE: игрок выбирает attack/defense/position → +25 к приоритету AI на 4 раунда

PvP: оба игрока получают совет, принимают/отклоняют → принявший получает эффект adrenaline на 4 раунда

```
// pvpCombatEngine.js:393-396
```

```

applyCoachAdvice(player) {
    player.activeEffects.push({ type: 'adrenaline', roundsLeft: COACH_BOOST_ROUNDS }); // 4
}

```

Совет зависит от HP:

```

generateCoachAdvice(player) {
    if (player.hp <= EMERGENCY_HP_THRESHOLD) { // <= 30
        return { type: 'use_dice', message: 'coach_advice_low_hp' };
    }
    return { type: 'keep_fighting', message: 'coach_advice_keep_going' };
}

```

// p

Серверная симуляция — что считает сервер

Сервер (PvPCombatEngine) полностью управляет боем:

1. HP обоих игроков — авторитативно на сервере
2. Урон — считается из moves.js на сервере
3. Speed / порядок ударов — определяется сервером
4. Dice эффекты — применяются и тикают на сервере
5. Результат — определяется сервером
6. ELO рейтинг — обновляется сервером

Клиент получает только результаты через WebSocket и отображает их.

Speed-based KO (одинаково)

```

// PvPCombatEngine.js:190-200
if (firstAttacker === 'player1') {
    this.player2.hp = Math.max(0, this.player2.hp - firstDamage);
    if (this.player2.hp > 0) {
        this.player1.hp = Math.max(0, this.player1.hp - secondDamage);
    }
} else { /* аналогично */ }

```

Как и в PvE — быстрый бьёт первым, может нокаутировать до ответного удара.

2. WebSocket протокол боя

Полная последовательность сообщений

КЛИЕНТ	СЕРВЕР	КЛИЕНТ
P1		P2
-- MatchmakingStartMsg --->		
	<--- MatchmakingQueueMsg	
	MatchmakingStartMsg --	
	MatchmakingQueueMsg --->	
	[матч найден]	

```

|
|<-- MatchFoundMsg -----
|
|          ----- MatchFoundMsg -->
|
|
|-- pvp_ready (deck) --->
|
|          pvp_ready (deck) --|
|
|          [оба ready → start]
|
|<-- fight_start -----
|
|          ----- fight_start -->
|
|          [COUNTDOWN_MS = 3000ms]
|
|          === ЦИКЛ РАУНДОВ (1..TOTAL_ROUNDS) ===
|
|<-- dice_available ----- (если кулдаун прошёл)
|
|          ----- dice_available -->
|
|-- dice_roll ----->
|<-- dice_rolled -----
|
|          [simulateRound]
|
|<-- round_result -----
|
|          ----- round_result -->
|
|          [ROUND_ANIMATION_MS = 1500ms]
|
|          === РАУНД >= COACH_MIN_ROUND (6) ===
|
|<-- coach_pause -----
|
|          ----- coach_pause -->
|
|-- coach_choice ----->
|
|          coach_choice -----|
|
|          [оба ответили ИЛИ таймаут]
|
|<-- coach_result -----
|
|          ----- coach_result -->
|
|          === КОНЕЦ БОЯ ===
|
|<-- fight_end -----
|
|          ----- fight_end -->

```

Обработка в pvpHandler.js

```

// pvpHandler.js:13-68 – три типа сообщений
switch (data.type) {
  case 'pvp_ready':    // Игрок готов, отправляет колоду
  case 'dice_roll':    // Игрок бросает кубик
  case 'coach_choice': // Игрок принимает/отклоняет коуча
}

```

△ В handler.js:118 для PvP сообщений используется dice_choice, но в pvpHandler.js:53 обрабатывается dice_roll. Расхождение в именовании — handler.js маршрутизирует dice_choice в pvpHandler, но pvpHandler ожидает dice_roll.

pvp_ready — старт боя

```
// pvpHandler.js:15-50
case 'pvp_ready': {
  // Привязать колоду и сокет к игроку
  match.player1.deck = data.deck;    // [{id, level}, ...]
  match.player1.socket = ws;
  match.player1.ready = true;
  // Когда оба ready → match.start()
  if (match.player1.ready && match.player2.ready) {
    match.start();
  }
}
```

Таймауты

Константа	Значение	Где используется
`COUNTDOWN_MS`	3000 ms	Пауза перед первым раундом (`pvpCombatEngine.js:73-75`)
`ROUND_ANIMATION_MS`	1500 ms	Пауза между раундами (`pvpCombatEngine.js:228-230`)
`DICE_PAUSE_TIMEOUT_MS`	10000 ms	△ **Объявлен в config.js, но НЕ используется** в pvpCombatEngine — dice бросается мгновенно без паузы
`COACH_PAUSE_TIMEOUT_MS`	10000 ms	Таймаут ожидания выбора обоих игроков (`pvpCombatEngine.js:344-348`)

Дисконнект

```
// pvpCombatEngine.js:441-465
onPlayerDisconnect(odId) {
  this.status = 'finished';
  // Победитель = оставшийся игрок
  const winner = odId === this.player1.odId ? this.player2 : this.player1;
```

```
// Отправить fight_end с reason: 'opponent_disconnected'
this.sendToPlayer(winner, 'fight_end', { ...result, reason: 'opponent_disconnected' });
this.saveFightResult(result);
}
```

```
// pvpHandler.js:71-77
function handlePvPDisconnect(odId) {
  const match = pvpMatchManager.getMatchByPlayer(odId);
  if (match && match.status !== 'finished') {
    match.onPlayerDisconnect(odId);
    pvpMatchManager.removeMatch(match.matchId);
  }
}
```

Вызывается из handler.js:73 при событии ws.on('close').

3. Управление матчем (pvpMatchManager)

Структура

```
// pvpMatchManager.js — Singleton
class PvPMatchManager {
  activeMatches: Map<matchId, PvPCombatEngine>
}
module.exports = new PvPMatchManager();
```

Жизненный цикл матча

1. createMatch(matchId, player1, player2)
 - new PvPCombatEngine(matchId, player1, player2)
 - activeMatches.set(matchId, engine)
 - status = 'waiting'
2. pvp_ready от обоих игроков
 - привязка deck и socket
 - player.ready = true
 - когда оба ready → engine.start()
 - status = 'running'
3. Цикл раундов (автоматический, по таймеру)
 - nextRound() каждые ROUND_ANIMATION_MS
 - simulateRound() → emit('round_result')
4. Coach пауза (раунд >= 6, один раз)
 - status = 'paused_coach'
 - ожидание coach_choice от обоих (или таймаут 10с)
 - resolveCoachPause() → status = 'running'
5. Завершение
 - endFight() → status = 'finished'
 - saveFightResult() → Prisma → PostgreSQL
 - pvpMatchManager.removeMatch(matchId)

Состояние матча на сервере

```
// PvPCombatEngine — полное состояние
{
  matchId: string,
  status: 'waiting' | 'running' | 'paused_coach' | 'finished',
  currentRound: number,
  maxRounds: TOTAL_ROUNDS (12),
  roundResults: [],
  pauseTimer: setTimeout ref,
  pendingChoices: { player1: null, player2: null },

  player1/player2: {
    odId: string,
    username: string,
    deck: [{id, level}, ...],
    hp: number,
    diceUsedRound: number,      // для кулдауна dice
    coachUsed: false,
    coachTriggered: false,
    activeEffects: [{type, roundsLeft}, ...],
    socket: WebSocket,
    ready: boolean,
  }
}
```

Персистенция результата (saveFightResult)

```
// pvpCombatEngine.js:469-561
await prisma.fight.create({
  data: {
    mode: 'pvp',
    matchId, fighterOneId, fighterTwoId,
    player1Id, player2Id,
    player1Hp, player2Hp,
    winner, winnerId,
    reason: 'normal' | 'disconnect',
    rounds, roundLog,
    isCompleted: true,
  },
});
```

ELO рейтинг

```
// pvpCombatEngine.js:579-594
calculateElo(winnerRating, loserRating, isDraw = false) {
  const K = 32;
  const expected = 1 / (1 + Math.pow(10, (loserRating - winnerRating) / 400));
  // Победа:
  winnerNew = Math.round(winnerRating + K * (1 - expected));
  loserNew = Math.round(loserRating + K * (0 - (1 - expected)));
  // Ничья:
  winnerNew = Math.round(winnerRating + K * (0.5 - expected));
```

```
loserNew = Math.round(loserRating + K * (0.5 - (1 - expected)));
}
```

Стандартная формула ELO с K=32, начальный рейтинг 1000.

XP за PvP

```
// pvpCombatEngine.js:563-577
const BASE_XP = 5;
const WIN_BONUS = 5;
const DRAW_BONUS = 2;
// Победа: 10 XP, Поражение: 5 XP, Ничья: 7 XP
```

Обновление статистики

При завершении боя обновляются поля в User:

- rating (ELO)
- pvpTotalFights (+1)
- pvpWins / pvpLosses / pvpDraws (+1)
- totalFights (+1)
- wins / losses / draws (+1)

4. Клиентский Vuex-модуль (pvpState)

Состояние

```
{
  currentPvPFight: null, // объект текущего боя
  pvpStats: {
    rating: 1000,
    wins: 0,
    losses: 0,
    draws: 0,
  },
  status: 'idle', // idle | searching | in_fight | finished
  currentMatchId: null,
  pvpFightStatus: 'idle', // idle | ready | fighting | paused | finished
  opponentInfo: null,
  isPlayer1: false,
}
```

Лиги (рейтинговые)

```
// pvpState.js:83-91
if (rating >= 3000) → Champion
if (rating >= 2500) → Diamond
if (rating >= 2000) → Platinum
if (rating >= 1500) → Gold
```

```
if (rating >= 1000) → Silver
else → Bronze
```

⚠ Расхождение фронт/бэкенд: расчёт рейтинга

Клиент (pvpState.js:40-53):

```
const bonus = Math.round(diff / 20);
return Math.max(10, base + bonus); // победа: min +10
return Math.min(-10, -base + penalty); // поражение: max -10
```

Сервер (pvpCombatEngine.js:579-594):

⚠ Разные формулы — клиент использует упрощённую линейную формулу, сервер — стандартную ELO. Серверная является авторитативной.

⚠ Моск-данные противника

pvpState.js:22-37 содержит generateOpponentFighter с захардкоженными mock-данными:

```
deck: [{ id: 'jab', damage: 8, speed: 9 }, { id: 'cross', ... }] // ⚠ 'cross' не существует в moves.js
```

Это legacy-код для fallback PvP (без серверного матча). В server-driven PvP (pvpMatchId) эти данные не используются.

localStorage

Ключ: hexlash_pvp

Сохраняется: pvpStats и currentPvPFight.

При загрузке приложения восстанавливаются из localStorage, затем перезаписываются данными с сервера через restoreFromServer.

Раздел 4 — Матчмейкинг и челленджи

Матчмейкинг и челленджи

Документация по подбору соперников и системе вызовов друзей.

Файлы:

- backend/src/services/matchmaking.js — сервис матчмейкинга
- backend/src/websocket/handler.js — WebSocket: challenge_send/accept/decline, matchmaking
- src/core/state/modules/friendsState.js — Vuex-модуль друзей и челленджей
- src/components/pvp/ChallengeNotification.vue — UI входящего челленджа

1. Матчмейкинг

Алгоритм подбора

По рейтингу. ⚠ Гео-матчмейкинг не реализован — нет никакого учёта региона/пинга.

```
// matchmaking.js:76-110 - tryFindMatch
for (const [oppId, opponent] of this.queue) {
  if (oppId === odId) continue;
  const ratingDiff = Math.abs(player.rating - opponent.rating);
  const maxRange = Math.max(player.searchRange, opponent.searchRange);
  if (ratingDiff <= maxRange && ratingDiff < bestDiff) {
    bestMatch = opponent;
    bestDiff = ratingDiff;
  }
}
```

Алгоритм:

1. Перебор всех игроков в очереди
2. Сравнение разницы рейтингов с допустимым диапазоном
3. Выбирается ближайший по рейтингу противник
4. Диапазон = max(searchRange игрока, searchRange противника)

Константы матчмейкинга

Константа	Значение	Описание
`SEARCH_RANGE_INITIAL`	300	Начальный диапазон поиска по рейтингу
`SEARCH_RANGE_STEP`	100	Расширение диапазона за шаг
`SEARCH_RANGE_MAX`	1000	Максимальный диапазон

`SEARCH_EXPAND_INTERVAL_MS`	5000	Интервал расширения (5 сек)
`SEARCH_TIMEOUT_MS`	120000	Таймаут поиска (2 мин)

Расширение поиска со временем

```
0с → диапазон 300 (±300 рейтинга)
5с → диапазон 400
10с → диапазон 500
15с → диапазон 600
...
35с → диапазон 1000 (максимум)
...
120с → таймаут, игрок удаляется из очереди
```

Расширение реализовано через setInterval для каждого игрока:

```
const expandTimer = setInterval(() => {
  p.searchRange = Math.min(p.searchRange + SEARCH_RANGE_STEP, SEARCH_RANGE_MAX);
  if (Date.now() - p.searchingSince > SEARCH_TIMEOUT_MS) {
    this.removeFromQueue(player.odId);
  }
}, SEARCH_EXPAND_INTERVAL_MS);
```

Периодическая проверка

Помимо проверки при добавлении в очередь, сервер каждые 3 секунды пытается найти пары:

```
// handler.js:589-596
setInterval(() => {
  for (const [odId] of matchmaking.queue) {
    const match = matchmaking.tryFindMatch(odId);
    if (match) notifyMatch(match);
  }
}, 3000);
```

Данные в очереди

```
// matchmaking.js:25-33
entry = {
  odId: string,           // ID пользователя
  username: string,
  rating: number,         // default 1000
  skin: string | null,
  avatarUrl: string | null,
  searchRange: 300,       // расширяется со временем
  searchingSince: Date.now(),
}
```

WebSocket протокол матчмейкинга

Клиент	Сервер
-- MatchmakingStartMsg ----->	
{ matchmakingRequest:	
{ username, rating,	
skin, avatarUrl } }	
<-- MatchmakingQueueMsg -----	
{ queueSize: N }	
[ожидание... расширение]	
<-- MatchFoundMsg -----	
{ matchId, opponent:	
{ odId, username, rating,	
skin, avatarUrl } }	
ИЛИ	
-- MatchmakingCancelMsg ----->	
<-- MatchmakingCancelledMsg -----	

Создание матча при нахождении пары

```
// matchmaking.js:113-136
createMatch(player1, player2) {
  const matchId = `match_${Date.now()}_${random}`;
  // Удалить обоих из очереди
  this.removeFromQueue(player1.odId);
  this.removeFromQueue(player2.odId);
  // Зарегистрировать матч (колоды пока пустые — придут через rvp_ready)
  rvpMatchManager.createMatch(matchId, {
    odId: player1.odId, username: player1.username, deck: []
  }, {
    odId: player2.odId, username: player2.username, deck: []
  });
}
```

⚠ Колоды (deck) при создании матча пустые — они передаются позже через rvp_ready.

Что реализовано vs заявлено

Фича	Статус
Подбор по рейтингу	Реализован
Расширение диапазона поиска	Реализован
Таймаут 2 мин	Реализован
Гео-матчмейкинг	⚠ Не реализован
Ping-based матчмейкинг	⚠ Не реализован

Учёт уровня карт / power rating	△ Не реализован – только ELO рейтинг
---------------------------------	--------------------------------------

2. Челленджи (друзья)

Полный флоу

Игрок А (отправитель)	Сервер	Игрок В (получатель)
-- challenge_send ----->		
{ targetUserId, username,		
rating }		
	-- challenge_received ----->	
	{ from: { odId, username,	
	rating }, challengeId }	
<-- challenge_sent -----		
{ targetUserId }		
[10 секунд на ответ]		
	<-- challenge_accepted -----	
	{ challengerOdId,	
	challengerUsername,	
	challengerRating }	
[создание матча через pvpMatchManager]		
<-- challenge_start -----		
{ matchId, opponent }	-- challenge_start ----->	
	{ matchId, opponent }	
[оба навигируются на /fight?mode=pvp&matchId=...]		

Отклонение

Игрок В	Сервер	Игрок А
-- challenge_declined --->		
{ challengerOdId }		
	-- challenge_declined -->	
	{ declinedBy }	

Таймер 10 секунд

На клиенте отправителя (friendsState.js:250-256):

```
challengeTimeout = setTimeout(() => {
```

// A

```

    if (s.challenge.outgoing?.odId === friend.id) {
      commit('clearOutgoingChallenge');
    }
  }, 10000);

```

Также в `sendChallenge`: `expiresAt: Date.now() + 10000`

На клиенте получателя (**ChallengeNotification.vue:41,108-116**):

```

timer.value = CHALLENGE_DURATION;

function startTimer() {
  timerInterval = setInterval(() => {
    timer.value--;
    if (timer.value <= 0) {
      declineChallenge(); // автоматический отказ
    }
  }, 1000);
}

```

⚠ На сервере таймера нет — сервер не отслеживает истечение челленджа. Таймер только на клиентах.

Проверка онлайн-статуса

```

// handler.js:414-426
function handleChallengeSend(ws, userId, msg) {
  const targetSocket = clients.get(targetUserId);
  if (!targetSocket) {
    sendMessage(ws, {
      type: 'challenge_error',
      message: 'friend_offline',
    });
    return;
  }
  // ...
}

```

Проверка: есть ли WebSocket-соединение у целевого игрока в `clients Map`.

При ассепт тоже проверяется:

```

const challengerSocket = clients.get(challengerOdId);
if (!challengerSocket) {
  sendMessage(ws, { type: 'challenge_error', message: 'challenger_offline' });
  return;
}

```

Создание матча при ассепт

```

// handler.js:466-498
// 1. Получить данные принявшего из БД
const acceptor = await prisma.user.findUnique({ where: { id: userId } });

// 2. Создать матч (колоды пустые — придут через pvp_ready)
const matchId = `match_${Date.now()}_${random}`;

```

```
pvpMatchManager.createMatch(matchId,
  { odId: challengerOdId, username: challengerUsername, deck: [] },
  { odId: userId, username: acceptorUsername, deck: [] }
);

// 3. Отправить challenge_start обоим
// Challenger:
sendMessage(challengerSocket, {
  type: 'challenge_start',
  matchId,
  opponent: { odId: userId, username: acceptorUsername, rating: acceptorRating }
});
// Acceptor:
sendMessage(ws, {
  type: 'challenge_start',
  matchId,
  opponent: { odId: challengerOdId, username: challengerUsername, rating: challengerRating }
});
```

Навигация на бой

Получатель (acceptor) — в ChallengeNotification.vue:53-67:

```
const data = event.detail;
store.commit('pvp/SET_PVP_MATCH', {
  matchId: data.matchId,
  opponent: data.opponent,
  isPlayer1: false,      // acceptor = player2
});
router.push({
  path: '/fight',
  query: { mode: 'pvp', matchId: data.matchId },
});
}
```

Отправитель (challenger) — аналогичная логика в обработке challenge_start события (в websocketState или AppVue), навигация на /fight?mode=pvp&matchId=... с isPlayer1: true.

Ошибки челленджа

Событие	Сообщение	Причина
`challenge_error`	`friend_offline`	Целевой игрок не подключён к WS
`challenge_error`	`challenger_offline`	Отправитель отключился до accett
`challenge_error`	`match_creation_failed`	Ошибка при создании матча в БД

UI ChallengeNotification

- Позиция: position: fixed; top: 0; z-index: 9999
- Анимация: slide-down (300ms вход, 200ms выход)
- Содержимое: иконка мечей (SVG), имя + рейтинг вызывающего, таймер в секундах
- Кнопки: Accept (розовый) / Decline (серый)
- Слушает события: challenge-received, challenge-start, challenge-sent, challenge-declined-response, challenge-error

Предусловия отправки челленджа

```
// friendsState.js:224-258
sendChallenge({ ... }, friend) {
  if (friend.status === 'offline') return false; // друг офлайн
  if (s.challenge.outgoing) return false;       // уже есть активный вызов
  // ...
}
```

Раздел 5 — Auto Fight

Auto Fight — автоматические бои

Документация по механике автоматических боёв.

Файлы:

- src/core/state/modules/autoFightState.js — Vuex-модуль автобоя
- src/components/fragments/fight/AutoFightToggle.vue — кнопка вкл/выкл
- src/components/fragments/fight/AutoFightStatus.vue — статус и таймер
- src/core/constants.js — константы AUTO_FIGHT_*

1. Механика Auto Fight

Константы

Константа	Значение	Описание
`AUTO_FIGHT_MIN_INTERVAL`	3 600 000 мс (60 мин)	Минимальный интервал между боями
`AUTO_FIGHT_MAX_INTERVAL`	3 600 000 мс (60 мин)	Максимальный интервал между боями
`AUTO_FIGHT_MAX_PER_DAY`	24	Лимит боёв в день
`AUTO_FIGHT_MAX_PER_SESSION`	48	Лимит боёв за сессию

△ MIN_INTERVAL === MAX_INTERVAL (оба 60 мин) — интервал фиксированный, рандомизации фактически нет:

```
function getRandomInterval() {
  return Math.random() * (MAX_INTERVAL - MIN_INTERVAL) + MIN_INTERVAL;
  // = Math.random() * 0 + 3600000 = 3600000 (всегда 60 мин)
}
```

// а

Включение (AutoFightToggle)

```
// AutoFightToggle.vue:21-27
const toggle = () => {
  if (isEnabled.value) {
    store.dispatch('autoFight/disable');
  } else {
    store.dispatch('autoFight/enable');
  }
};
```

Предусловие: isBuildValid — все 3 модуля должны быть выбраны (canToggle).

Action enable (autoFightState.js:303-318):

```
const modules = rootGetters['fight/getPlayerModules'];
if (!modules || !modules.every(m => m !== null)) return false;

commit('setEnabled', true);
commit('setEnabledAt', Date.now());
commit('setNextFightAt', Date.now() + getRandomInterval()); // +60 мин
commit('setStoppingAfterCurrent', false);
saveState(state);

// Запрос разрешения на push-уведомления
if ('Notification' in window && Notification.permission === 'default') {
  Notification.requestPermission();
}
return true;
}
```

Выключение

Два варианта:

1. Мягкое (disable) — если идёт бой, ставит флаг stoppingAfterCurrent = true, после завершения текущего боя выключается
2. Жёсткое (forceStop) — немедленная остановка, сброс liveFight

```
// autoFightState.js:322-339
disable({ commit, state }) {
  if (state.liveFight) {
    commit('setStoppingAfterCurrent', true); // дождаться окончания
  } else {
    commit('setEnabled', false);
    commit('setNextFightAt', null);
  }
}

forceStop({ commit, state }) {
  commit('setEnabled', false);
  commit('setNextFightAt', null);
  commit('setStoppingAfterCurrent', false);
  commit('setLiveFight', null);
}
```

Как симулируется бой

Используется тот же движок что и для PvE — CombatEngine.resolveRoundLive + ModuleAIStrategy.

Функция simulateFullFight (autoFightState.js:66-157):

1. OpponentGenerator.generate(difficulty, playerPower)
2. Создать AI: ai1 (игрок), ai2 (противник)
3. Цикл раундов (1..TOTAL_ROUNDS):
 - a. ai.selectAction(hp, MAX_HP, isOverdrive)
 - b. CombatEngine.getMoveInfo(round, playerDeck, ..., opponentDeck, ...)
 - c. CombatEngine.resolveRoundLive(...)

- d. Модификаторы сбрасываются каждый раунд
 - e. Emergency: автоматически medkit при HP < 30%
 - f. Coach: автоматически (HP < 50 → defense, иначе → attack)
4. Определение результата: win / lose / draw

△ Отличия от ручного боя:

- Emergency всегда medkit (нет выбора протокола)
- Coach стратегия: HP < 50 → defense, иначе → attack
- Dice не бросается в simulateFullFight (△ в офлайн-симуляции dice полностью отсутствует)

Проверка лимитов

```
// autoFightState.js:198-202
canStartAutoFight: (s) => {
  if (s.fightsToday >= AUTO_FIGHT_MAX_PER_DAY) return { allowed: false, reason: 'dailyLimit' };
  if (s.sessionFights >= AUTO_FIGHT_MAX_PER_SESSION) return { allowed: false, reason: 'sessionLimit' };
  return { allowed: true };
}
```

2. Офлайн и catch-up

Что происходит когда вкладка закрыта

Автобой не работает в реальном времени в фоне. Когда пользователь возвращается, пропущенные бои досимулируются мгновенно.

Catch-up (checkAndRunPending)

Вызывается:

- При монтировании AutoFightStatus (onMounted)
- Каждую секунду через setInterval в AutoFightStatus (когда now >= nextFightAt)
- При visibilitychange (возврат на вкладку)

```
// autoFightState.js:346-455
checkAndRunPending({ ... }) {
  // 1. Проверка дневного ресета
  // 2. Проверка валидности модулей
  // 3. Проверка, нет ли активного ручного боя

  // 4. Досимуляция пропущенных боёв:
  let nextAt = state.nextFightAt;
  while (nextAt && now >= nextAt
    && state.fightsToday < AUTO_FIGHT_MAX_PER_DAY
    && state.sessionFights < AUTO_FIGHT_MAX_PER_SESSION) {

    const fightData = simulateFullFight(modules, difficulty, playerPower, deck, cardLevels);

    // Логирование
    commit('addFightToLog', logEntry);
    commit('incrementStats', fightData.result);
  }
}
```

```

    commit('addExp', expGain);

    // Начисление XP
    dispatch('progression/onFightEnd', { result }, { root: true });

    // Синхронизация с сервером
    apiClient.post('/fight/save', { isWin, isDraw, roundsPlayed, totalDamageDealt: 0 });

    // Push-уведомление
    dispatch('sendNotification', { fight: logEntry });

    nextAt = nextAt + getRandomInterval(); // +60 мин
  }

  commit('setNextFightAt', nextAt); // всегда в будущем
}

```

Пример: Вкладка была закрыта 3 часа. При возврате:

- nextFightAt было 3 часа назад
- Цикл while пройдёт 3 итерации (3 боя по 60 мин)
- Все 3 боя симулируются мгновенно
- nextFightAt ставится в будущее (+60 мин от последнего)

XP за автобой

```

const expGain = fightData.result === 'win' ? 10 : 5;
// ▲ Ничья тоже даёт 5 XP (нет отдельной проверки на draw)

```

Push-уведомления

```

// autoFightState.js:515-531
sendNotification(_, { fight }) {
  if (!('Notification' in window) || Notification.permission !== 'granted') return;
  new Notification('Hexlash Auto Fight', {
    body: fight.result === 'win'
      ? `Victory vs ${fight.opponent}! +${xp} XP`
      : fight.result === 'lose'
        ? `Defeat vs ${fight.opponent}. +${xp} XP`
        : `Draw vs ${fight.opponent}. +${xp} XP`,
    icon: '/favicon.ico',
    tag: 'autofight-' + fight.id, // предотвращает дубликаты
  });
}

```

Используется Web Notification API. Разрешение запрашивается при включении автобоя.

3. Персистентность

localStorage: hexlash_autofight_state

```
// autoFightState.js:20-34
{
  enabled: boolean,
  enabledAt: number | null,      // timestamp включения
  nextFightAt: number | null,    // timestamp следующего боя
  stoppingAfterCurrent: boolean,
  difficulty: string,            // 'easy' | 'medium' | 'hard'
  fightsToday: number,
  lastFightDate: string | null,  // "YYYY-MM-DD"
  wins: number,
  losses: number,
  draws: number,
  totalExpGained: number,
  sessionFights: number,
}
```

localStorage: hexlash_autofight_history

```
// autoFightState.js:44-48
// Массив последних 100 записей боёв:
[
  {
    id: 'autofight_' + timestamp,
    timestamp: number,
    opponent: string,           // имя противника
    opponentSkin: string,
    result: 'win' | 'lose' | 'draw',
    rounds: number,
    hp1Final: number,
    hp2Final: number,
    expGained: number,          // 10 (win) или 5 (lose/draw)
  }
]
```

Лимит: 100 записей (fightLog.slice(-100)).

Синхронизация с сервером

```
// autoFightState.js:425-430
apiClient.post('/fight/save', {
  isWin: fightData.result === 'win',
  isDraw: fightData.result === 'draw',
  roundsPlayed: fightData.rounds,
  totalDamageDealt: 0,          // ▲ всегда 0 — урон не передаётся
}, { authRequired: true }).catch(() => {}); // ошибки игнорируются
```

На сервере POST /fight/save инкрементирует:

- pveWins / pveLosses / pveDraws (+1)
- pveTotalFights (+1)

▲ Fire-and-forget: ошибки при отправке на сервер игнорируются (.catch(() => {})). Если сервер недоступен — бои считаются только локально.

4. Дневной ресет

Когда срабатывает

При вызове `init` (загрузка приложения) или `checkAndRunPending` (каждый тик):

```
// autoFightState.js:283-296 (init) и 351-361 (checkAndRunPending)
const today = getTodayDate(); // "YYYY-MM-DD"
if (state.lastFightDate && state.lastFightDate !== today) {
  commit('setFightsToday', 0);
  commit('setWins', 0);
  commit('setLosses', 0);
  commit('setDraws', 0);
  commit('setTotalExpGained', 0);
  commit('setLastFightDate', today);
  commit('setFightLog', []); // очистка лога
  saveHistory([]);           // очистка localStorage
}
```

Что сбрасывается

Поле	Сбрасывается	Новое значение
<code>`fightsToday`</code>	Да	0
<code>`wins`</code>	Да	0
<code>`losses`</code>	Да	0
<code>`draws`</code>	Да	0
<code>`totalExpGained`</code>	Да	0
<code>`lastFightDate`</code>	Обновляется	Сегодняшняя дата
<code>`fightLog`</code>	Очищается	<code>[]</code>
<code>`enabled`</code>	**Нет**	Остаётся как есть
<code>`sessionFights`</code>	**Нет**	Остаётся как есть

Ручная кнопка сброса

⚠ Ручной кнопки сброса нет. Дневной ресет происходит только автоматически при смене даты. В UI есть только кнопка "View Fight Log" — нет кнопки очистки лога или сброса счётчиков.

Есть action `clearHistory`, но он не вызывается ни из какого UI-компонента:

```
commit('setFightLog', []);
saveHistory([]);
}
```

5. AutoFightStatus UI

Что показывает

Компонент `AutoFightStatus.vue` — карточка состояния автобоя:

Элемент	Данные	Источник
Заголовок	"AUTO FIGHT ACTIVE" (i18n)	`t.autoFight.lblAutoFightActive`
Таймер	`MM:SS` до следующего боя	Вычисляется из `nextFightAt - now`
Бои сегодня	Число	`fightsToday`
Победы	Зелёное число	`wins`
Поражения	Красное число	`losses`
"Останавливается..."	Жёлтый мигающий текст	`stoppingAfterCurrent`
Кнопка "View Fight Log"	Навигация на `/arena/autofight-log`	—

Обновление в реальном времени

```
// AutoFightStatus.vue:55-71
onMounted(() => {
  // Проверка пропущенных боёв при монтировании
  if (isEnabled.value) {
    store.dispatch('autoFight/checkAndRunPending');
  }

  // Таймер обновления каждую секунду
  timerInterval = setInterval(() => {
    now.value = Date.now();

    // Триггер автобоя когда таймер дошёл до 0
    if (isEnabled.value && nextFightAt.value && now.value >= nextFightAt.value && !fightTriggered) {
      fightTriggered = true;
      store.dispatch('autoFight/checkAndRunPending')
        .then(() => { fightTriggered = false; });
    }
  }, 1000);
});
```

Формат таймера

```
// AutoFightStatus.vue:83-90
const timeDisplay = computed(() => {
  if (!nextFightAt.value) return '--:--';
  const diff = Math.max(0, Math.ceil((nextFightAt.value - now.value) / 1000));
  if (diff <= 0) return '--:--';
  const m = Math.floor(diff / 60);
  const s = diff % 60;
  return `${m}:${s.toString().padStart(2, '0')}`;
});
```

Пример: 59:42, 0:05, --:-- (нет следующего боя).

Защита от двойного срабатывания

```
let fightTriggered = false;  
// Ставится true перед dispatch, false после завершения  
// watch на nextFightAt сбрасывает флаг при новом расписании  
watch(nextFightAt, () => { fightTriggered = false; });
```

Блокировка при ручном бое

Если идёт ручной бой (`fightPhase === 'fighting' || 'coach'`), автобой не запускается — `checkAndRunPending` просто переносит `nextFightAt` в будущее:

```
// autoFightState.js:375-383  
const fightPhase = rootGetters['fight/getFightPhase'];  
if (fightPhase === 'fighting' || fightPhase === 'coach') {  
  if (state.nextFightAt && now >= state.nextFightAt) {  
    commit('setNextFightAt', now + getRandomInterval());  
  }  
  return;  
}
```

Раздел 6 — Приложения

6.1 Таблица всех приёмов (18 шт.)

Speed (Скорость)

ID	Имя	Damage (1→5)	Speed (1→5)	Base Power
jab	Джэб	8/10/12/15/18	1.20/1.30/1.40/1.50/1.60	8
double_jab	Двойной джэб	12/15/18/22/26	1.10/1.20/1.30/1.40/1.50	10
rapid_fire	Скорострел	15/18/22/27/32	1.00/1.10/1.20/1.30/1.40	12
combo_strike	Комбо-удар	20/24/29/35/42	0.90/1.00/1.10/1.20/1.30	14
flurry	Шквал	25/30/36/43/52	0.80/0.90/1.00/1.10/1.20	16
hurricane	Ураган	32/38/46/55/66	0.70/0.80/0.90/1.00/1.10	18

Power (Сила)

ID	Имя	Damage (1→5)	Speed (1→5)	Base Power
straight	Прямой	12/15/18/22/27	0.80/0.85/0.90/0.95/1.00	10
hook	Хук	16/20/24/29/35	0.75/0.80/0.85/0.90/0.95	13
uppercut	Апперкот	20/25/30/36/44	0.70/0.75/0.80/0.85/0.90	16
haymaker	Свинг	26/32/38/46/56	0.60/0.65/0.70/0.75/0.80	19
hammer_fist	Молот	32/40/48/58/70	0.50/0.55/0.60/0.65/0.70	22
knockout_blow	Нокаут	42/52/62/75/90	0.40/0.45/0.50/0.55/0.60	25

Technique (Техника)

ID	Имя	Damage (1→5)	Speed (1→5)	Base Power
block_strike	Блок-удар	10/12/15/18/22	1.00/1.05/1.10/1.15/1.20	9
counter_jab	Контр-джэб	14/17/21/25/30	0.95/1.00/1.05/1.10/1.15	11
feint_cross	Обманный кросс	18/22/27/32/39	0.90/0.95/1.00/1.05/1.10	13
parry_punish	Парирование	22/27/33/40/48	0.85/0.90/0.95/1.00/1.05	15
slip_counter	Уклон-контр	28/34/41/50/60	0.80/0.85/0.90/0.95/1.00	17
precision_strike	Точный удар	35/43/52/63/76	0.75/0.80/0.85/0.90/0.95	20

6.2 Таблица эффектов Dice (PvE vs PvP)

Эффект	PvE (клиент)	PvP (сервер)
heal	+15 HP мгновенно	+20 HP мгновенно
adrenaline	attackMultiplier=2 (1 раунд)	x1.3 урона (2 раунда)
shield	Полный блок 1 атаки (1 раунд)	x0.5 входящего урона (2 раунда)
blind	Промех противника (1 раунд)	50% шанс промаха (2 раунда)
rage	-20 HP противнику мгновенно	x1.5 урона (2 раунда)
crit	-30 HP противнику мгновенно	x2 урона (1 раунд)

6.3 Таблица констант боя

Константа	Значение	Описание
-----------	----------	----------

MAX_HP	100	Стартовое HP
MAX_ROUNDS	10	Основные раунды
EXTRA_ROUNDS	2	Дополнительные раунды (Overdrive)
TOTAL_ROUNDS	12	MAX_ROUNDS + EXTRA_ROUNDS
EXTRA_ROUND_DAMAGE_MULTIPLIER	2	Множитель урона в Overdrive
COUNTDOWN	3	Секунды до боя
ROUND_ANIMATION_MS	1500	Длительность анимации раунда (мс)
BASE_DAMAGE	15	Базовый урон
POSITION_BONUS	5	Бонус от position
DODGE_CHANCE	0.12	Шанс уклонения (12%)
CRIT_CHANCE	0.10	Шанс крита (10%)
CRIT_MULT	1.5	Множитель крита
DICE_COOLDOWN_ROUNDS	3	Кулдаун dice
COACH_MIN_ROUND	6	Минимальный раунд для Coach
COACH_TRIGGER_CHANCE	1.0	Шанс срабатывания Coach (100%)
COACH_BOOST_ROUNDS	4	Длительность бонуса Coach
EMERGENCY_HP_THRESHOLD	30	Порог HP для Emergency (30%)
MIN_DECK_SIZE	4	Мин. размер колоды
MAX_DECK_SIZE	8	Макс. размер колоды
AUTO_FIGHT_MIN_INTERVAL	3600000	Интервал автобоя (60 мин)
AUTO_FIGHT_MAX_INTERVAL	3600000	Макс. интервал (60 мин)
AUTO_FIGHT_MAX_PER_DAY	24	Лимит автобоев в день
AUTO_FIGHT_MAX_PER_SESSION	48	Лимит автобоев за сессию
SEARCH_RANGE_INITIAL	300	Начальный диапазон матчмейкинга
SEARCH_RANGE_STEP	100	Расширение диапазона
SEARCH_RANGE_MAX	1000	Макс. диапазон
SEARCH_TIMEOUT_MS	120000	Таймаут поиска (2 мин)
DICE_PAUSE_TIMEOUT_MS	10000	Объявлен, но не используется
COACH_PAUSE_TIMEOUT_MS	10000	Таймаут выбора Coach (PvP)

6.4 Сводка предупреждений

Все предупреждения (отмечены символом $\triangle$), собранные из документации:

1. $\triangle$ Если защитник в position — есть дополнительный шанс уклонения:
2. $\triangle$ Модификаторы применяются только к Fighter 1 (игроку). Fighter 2 (AI) всегда атакует с mult = 1 и без shield/blind.
3. $\triangle$ shouldPickupDiceltem используется только в PvP-контексте (серверная логика). В PvE-бою dice — привилегия игрока.
4. $\triangle$ Emergency protocol не связан с колодой или модулями. Тип протокола задаётся через setEmergencyProtocol action и хранится в стейте. Выбор делается на экране подготовки к бою (PreparationView).
5. $\triangle$ При сохранении dice-анимация сбрасывается: activeltem всегда null, ready вычисляется из cooldownLeft.
6. $\triangle$ Досимулированные раунды не показывают dice/coach/emergency — используется _simulateOneRound, который не вызывает эти механики. Это значит: пропущенные раунды = чистый бой без спецмеханик.
7. ##### $\triangle$ Нет системы действий (attack/defense/position)
8. ##### $\triangle$ Другая формула урона
9. ##### $\triangle$ Dice эффекты отличаются
10. ##### $\triangle$ Coach работает иначе

11. ⚠ В `handler.js:118` для PvP сообщений используется `dice_choice`, но в `pvphandler.js:53` обрабатывается `dice_roll`. Расхождение в именовании — `handler.js` маршрутизирует `dice_choice` в `pvphandler`, но `pvphandler` ожидает `dice_roll`.
12. ⚠ Разные формулы — клиент использует упрощённую линейную формулу, сервер — стандартную ELO. Серверная является авторитативной.
13. По рейтингу. ⚠ Гео-матчмейкинг не реализован — нет никакого учёта региона/пинга.
14. ⚠ Колоды (`deck`) при создании матча пустые — они передаются позже через `pvpr_ready`.
15. ⚠ На сервере таймера нет — сервер не отслеживает истечение челленджа. Таймер только на клиентах.
16. ⚠ `MIN_INTERVAL === MAX_INTERVAL` (оба 60 мин) — интервал фиксированный, рандомизации фактически нет:
17. ⚠ Отличия от ручного боя:
18. Dice не бросается в `simulateFullFight` (⚠ в офлайн-симуляции `dice` полностью отсутствует)
19. ⚠ Fire-and-forget: ошибки при отправке на сервер игнорируются (`.catch(() => {})`). Если сервер недоступен — бои считаются только локально.
20. ⚠ Ручной кнопки сброса нет. Дневной ресет происходит только автоматически при смене даты. В UI есть только кнопка "View Fight Log" — нет кнопки очистки лога или сброса счётчиков.